

2.3 The Learning Rate

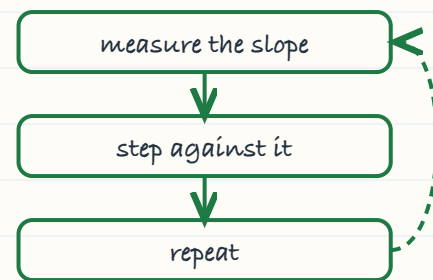
Study Notes · Third Eye Labs
Season 2, episode 3

THE ONE SENTENCE

The gradient says **WHICH WAY**.

The learning rate says **HOW FAR**.

That second number decides whether the model learns, barely learns, or destroys itself.



the loop — this is all training is

BEFORE YOU START — if you write software but not machine learning

You have already built this system — you just called it something else. A service that watches a metric and adjusts itself is an autoscaler. Training is the same control loop, with two differences: it adjusts 2,410 numbers instead of a replica count, and the metric it watches is how wrong the model is.

the ML word	what it actually is, in your terms
weight (or parameter)	A number the program tunes itself. Think of a config value that nobody sets by hand — the system searches for it. This network has 2,410 of them.
model	A function. Input goes in (64 pixel values), an answer comes out (which digit). The 2,410 weights are what make it give that answer rather than another.
loss (or score / cost)	One number for how wrong the model currently is — an error metric, like an error rate or p99 latency. Lower is better. Everything in training exists to reduce it.
gradient	For each weight: “if I nudge this one a hair, how much does the error move?” It is what you do by hand when perf-tuning — change one setting, measure, decide.
learning rate	How aggressively the loop changes each value per iteration. Exactly the knob that makes an autoscaler either flap or never react.
training	The loop: measure the error, nudge every weight a little in the direction that reduces it, repeat a few hundred times. That is the entire process.

If you have ever watched an autoscaler **thrash** because it reacted too hard, or sit useless because it reacted too softly — you already understand this episode. The learning rate is that dial.

1. The update, on one real weight

$$W1[7,11] = -0.2089$$

$$\text{gradient} = -0.0057$$

$$\text{lr} = 0.5$$

$$-0.2089 - 0.5 \times (-0.0057) =$$

$$\boxed{-0.2061}$$

General update (every weight)

$$\theta \leftarrow \theta - \eta \nabla L$$

θ = the weight

η = the learning rate (**HOW FAR**)

∇L = the arrow (**WHICH WAY**)

2,410 of these, every single step.

2. The experiment — one variable (only η changes)

Real network

64 → 32 → 10 (2,410 weights)

1,400 real handwritten digits

300 steps, same seed.

Held-out digits only — testing on what it memorised tells you nothing.

η	loss (start → end)	accuracy	verdict
0.5	2.3162 → 0.0718	12.6% → 97.2%	learns
0.002	2.3162 → 2.1622	12.6% → 28.2%	crawls — still useless
60	pinned at ceiling	12.6% → 8.3%	worse than guessing (10%)

⚠ "A smaller learning rate is safer" is the myth. It is how you wait all day and arrive nowhere.

3. Watch it actually learn

9

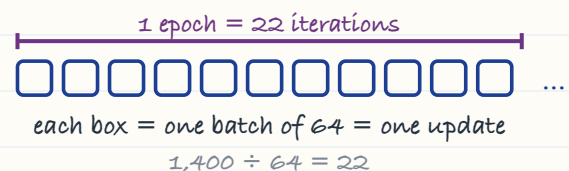
called 3 → 7 → 3 → 9 → 9
a real 9 · 99% confident

1

called 7 → 2 → 1 → 1 → 1
a real 1 · 60% confident

It learned. It did not become infallible — and knowing the difference is most of what using these models well means.

4. Epoch vs iteration vs batch



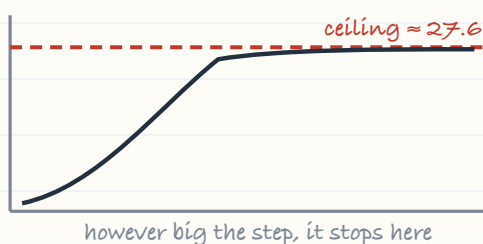
An epoch is NOT a unit of time. It is one pass over the data.

- 2.2 asked how WRONG a small batch makes your arrow (direction).
- 2.3 asks how MANY steps it buys you (counting).

5. Your loss will never reach NaN

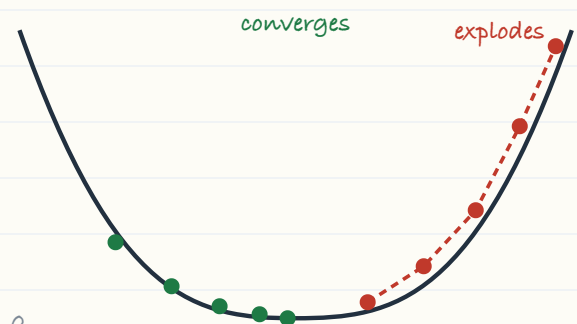
...if it is a classifier

lr	peak loss	NAN?
60	25.90	No
2,000	27.63	No
200,000	27.63	No



Where it really breaks: a loss with no ceiling (squared error). Each step multiplies the error by $\sim 29\times$:

6. The boundary is exact



below the stable step it always converges · above it, always explodes

For that surface the largest stable step is 0.701. Below it every run converges. Above it every run explodes. Not usually — always.

It comes from the shape of the surface itself, which is

0.0218 → 0.204 → 5.63 → 165 → 4,856 →

142,700 → ... → inf at step 209

△ It becomes inf, not NaN. Real NaN comes from arithmetic on infinities (inf-inf, 0×inf, log 0).

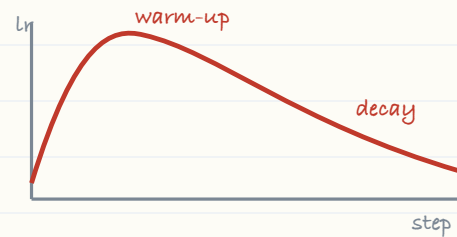
why nobody can hand you one learning rate that works for every model.

7. So: $lr = 3e-4$

```
optimizer = Adam(model.parameters(),  
lr=3e-4)
```

Three ten-thousandths of the arrow, every step. Small enough not to explode, big enough to actually move — and *nobody derived it*. Somebody ran the experiment above on a bigger model and that number worked.

8. Advanced — tools for the real world



- Schedules: change η over time. Warm-up → decay (step / cosine / exponential). Helps stability and final accuracy.
- Adaptive optimisers (Adam, RMSProp): a per-parameter step size, from past gradients. → 2.7
- Batch size interacts: bigger batch → less noise → you can usually afford a larger η .
- Diagnose it: watch loss, accuracy, gradient norm, update norm. Loss spikes or accuracy dropping → η too large. Nothing moving → too small.

NEXT: get the step size perfect and the front of a deep network can still learn nothing → 2.4, vanishing gradients.